

Dynamic Flow, Static Graph: KV Cache Reuse for Efficient LLM Serving on Mobile NPUs

Zhengxiang Huang
Shanghai Jiao Tong University
Shanghai, China

Yujie Sun
Shanghai Jiao Tong University
Shanghai, China

Chengfei Lv
Alibaba Group
Hangzhou, Zhejiang, China

Shengheng Chen
Shanghai Jiao Tong University
Shanghai, China

Zhaode Wang
Alibaba Group
Hangzhou, China

Fan Wu
Shanghai Jiao Tong University
Shanghai, China

Chaoyue Niu*
Shanghai Jiao Tong University
Shanghai, China

Zeyu Zhao
Shanghai Jiao Tong University
Shanghai, China

Guihai Chen
Shanghai Jiao Tong University
Shanghai, China

Abstract

On-device large language model (LLM) serving is a cornerstone of local-first personal intelligence, offering users data sovereignty, strong privacy guarantees, and freedom from cloud API latency and cost. Although KV caching is widely used to reduce latency in long-context inference, existing designs were primarily optimized for cloud GPUs with dynamic execution environments and abundant memory bandwidth. These architectural assumptions do not hold on mobile NPUs, where computation graphs must be statically compiled and both memory capacity and I/O bandwidth are severely constrained. In this work, we present a compute-storage co-design for mobile-centric prefix and non-prefix KV reuse. We first propose an intra-graph mechanism that maps selective KV recomputation onto static NPU graphs, reconciling algorithmic dynamicity with NPU staticity. We further develop an inter-graph scheduler to optimize chunk merging and minimize padding with dynamic programming. To address mobile bandwidth limitations, we introduce a hierarchical KV manager featuring a tree-hash-semantic hybrid structure, along with cost-aware prefetching and eviction policies. We also build a two-dimensional pipeline that overlaps KV loading, rerotation, and storage with NPU execution, hiding data-movement latency. Experiments across representative on-device workloads and LLMs show that our design reduces time-to-first-token (TTFT) by 40–60% compared with no reuse and prefix-only caching.

*Chaoyue Niu is the corresponding author (rvince@sjtu.edu.cn).



This work is licensed under a Creative Commons Attribution 4.0 International License.

EuroSys '27, Rabat, Morocco

© 2027 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2971-3/2027/04

<https://doi.org/10.1145/3842654.3848526>

CCS Concepts: • Human-centered computing → Ubiquitous and mobile devices; • Computing methodologies → Natural language processing.

Keywords: On-Device LLM Inference, Mobile NPU, KV Cache Reuse

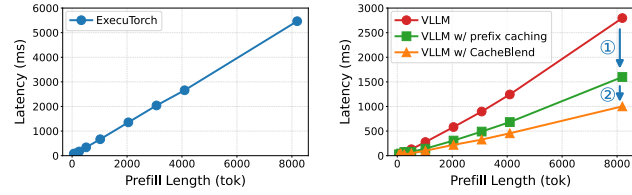
ACM Reference Format:

Zhengxiang Huang, Shengheng Chen, Chaoyue Niu, Yujie Sun, Zhaode Wang, Zeyu Zhao, Chengfei Lv, Fan Wu, and Guihai Chen. 2027. Dynamic Flow, Static Graph: KV Cache Reuse for Efficient LLM Serving on Mobile NPUs. In *22nd European Conference on Computer Systems (EuroSys '27)*, April 19–23, 2027, Rabat, Morocco. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/3842654.3848526>

1 Introduction

The rapid advancement of large language models (LLMs) is driving a fundamental shift in how intelligence is delivered to end users. Rather than relying solely on cloud-centric intelligence, driven by the imperatives of low cost, user privacy, offline availability, and low-latency interaction, leading technology companies are increasingly moving toward a local-first, user-centric intelligence paradigm that enables context-aware, continuously available assistance grounded in sensitive personal data. For example, Google Gemini emphasizes personalized intelligence experiences [2], while Google AI Edge Gallery [16] demonstrates practical deployment of generative models directly on smartphones; Apple Intelligence [5] adopts a hybrid device–cloud collaborative architecture [4], where an on-device 3B-scale LLM is responsible for privacy-sensitive and personalized tasks. Recent systems such as ClawMobile [9] and OmniInfer [40] further demonstrate the growing feasibility of deploying autonomous LLM agents directly on mobile devices.

On-device LLM serving forms the system foundation of the emerging user-centric, local-first intelligence stack. Like cloud services for personalized intelligence, mobile LLM services face significant challenges in handling *long-context*



(a) On-device prefill latency under different prefill length on Xiaomi 15, inference backed by ExecuTorch, using mobile NPU, 2–5× slower than cloud. (Qwen3-1.7B, batch size = 1) (b) Cloud prefill latency under different prefill length on RTX 5090. Reduction ① comes from prefix caching, while ② comes from non-prefix reuse. (Qwen3-1.7B, batch size = 16)

Figure 1. Long-context prefill latency on mobile devices and KV cache reuse for latency reduction in cloud LLM serving.

and *repeated-context* workloads. Such workload characteristics originate from how these applications construct their prompts, and are therefore shared by cloud and device LLM services alike [5, 9, 21, 26]. Consider several representative scenarios: a long-running conversational assistant whose prompt grows progressively with chat history; a document QA assistant that repeatedly retrieves the same set of user documents; a personal agent that loads shared skill descriptions or tool definitions for every invoked task. In each case, context grows long through accumulation or retrieval and becomes reusable because the same content recurs across requests. Large portions of the prompt, including system prompts, retrieved document chunks, and reusable skill instructions, are repeatedly reused across successive requests, introducing substantial redundancy during the transformer prefill phase. As illustrated in Figure 1(a), prefill latency, i.e., time-to-first-token (TTFT), increases rapidly with prompt length, reaching several seconds once prompts exceed 2k tokens and continuing to grow nearly linearly thereafter, severely degrading the interactive user experience. While the *long and repeated context* workload characteristic is dictated by the application layer rather than the hardware beneath, its cost is far higher on mobile. At the same prompt length, mobile NPUs are 2–5× slower than cloud GPUs (Figure 1(b)).

To avoid repeated prefill computation and reduce TTFT, previously computed Key-Value (KV) tensors for input tokens can be cached and reused when similar context appears again. Existing KV cache reuse designs (e.g., Mooncake [36], RAGCache [21], and LMCache [26]) mainly target *cloud-based deployment* scenarios, focusing on GPU inference and distributed storage architectures. These techniques have also been integrated into LLM serving engines such as VLLM [22] and SGLang [53]. Beyond prefix reuse, CacheBlend [50] further introduces selective KV recomputation to enable reuse of non-prefix KV segments. As shown in Figure 1(b), prefix caching and non-prefix KV reuse can reduce cloud serving latency by 2×–3× on cache hits for batched requests served on an RTX 5090 GPU.

However, KV cache reuse has not been well studied in on-device LLM serving scenarios. In particular, mobile LLM inference engines (e.g., ExecuTorch [12], MLC-LLM [30], llama.cpp [14], MediaPipe [11]) focus largely on efficient single-request execution through optimized kernels, heterogeneous CPU/GPU/NPU scheduling, and quantization. They typically provide little or no support for persistent KV reuse across requests as well as runtime hierarchical KV cache management, and essentially no support for quality-preserving non-prefix reuse. This gap is increasingly important: the workloads are exactly those that exhibit strong cross-request context repetition. Without reuse, mobile systems repeatedly recompute the same long contexts, wasting time, resources, and energy.

Directly porting cloud-based KV reuse techniques to mobile devices is fundamentally infeasible due to profound architectural differences between cloud and mobile hardware. First, from a compute perspective, mobile NPUs are architecturally distinct from cloud GPUs. Whereas cloud GPUs expose flexible, programmable tensor cores with SIMT parallelism that naturally accommodate dynamic tensor shapes and irregular computational graphs, mobile NPUs are built around fixed-shape matrix tile units that operate on statically pre-compiled computational graphs. Dynamic execution patterns, such as the variable token selection, input-dependent KV deviation computation, and runtime reuse-pattern variation, introduced by non-prefix selective recomputation, are fundamentally incompatible with this static execution model. Second, from a storage perspective, on-device platforms operate under memory bandwidth and capacity constraints that are orders of magnitude tighter than cloud deployments, creating a severe memory and bandwidth wall. Loading and storing large KV tensors can easily exceed the time saved by bypassing the prefill computation. Without a specialized hierarchical management and prefetching strategy that considers both NPU execution and flash I/O, the overhead of KV movement can negate the benefits of reuse. These differences are structural rather than transient: mobile SoCs remain constrained by stringent power and thermal budgets, and thus cannot scale compute and memory resources in the same manner as datacenter accelerators. Consequently, although long-context and context-reuse workloads are shared across device and cloud services, efficiently exploiting such reuse opportunities on mobile devices requires fundamentally novel engine-level designs rather than directly inheriting cloud-oriented mechanisms.

In this work, we build the first system that enables efficient prefix and non-prefix KV reuse for on-device LLM serving on mobile NPUs, as depicted in Figure 2. Unlike existing cloud designs that target cloud GPU kernels and distributed clusters, our key idea is a compute-storage co-design that adapts dynamic KV reuse algorithms to the static, quantized execution model of mobile NPUs while simultaneously respecting the severe bandwidth and capacity constraints of on-device

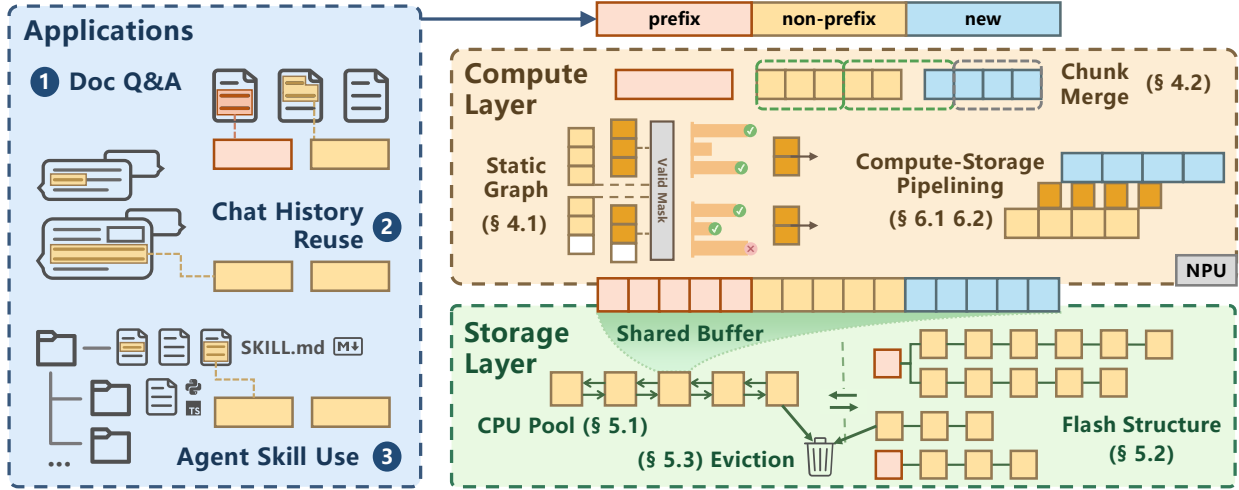


Figure 2. System overview of our on-device KV reuse design for long-context mobile LLM serving. The system jointly optimizes compute and storage for both prefix and non-prefix KV reuse, converting dynamic selective recomputation into efficient static NPU execution while enabling hierarchical KV management and asynchronous compute-storage pipeline overlap.

memory hierarchies. On the *compute side* (§ 4), to bridge the gap between dynamic algorithms and static hardware, we first map selective recomputation onto pre-compiled static NPU graphs, using chunk-and-pad, validity masking, and forced final-token selection to preserve irregular reuse semantics under fixed input shapes. At the inter-graph level, we formulate runtime execution as a multi-graph scheduling problem and develop a novel dynamic programming scheduler that merges invocations across a set of pre-compiled static graphs to minimize latency under dynamic input. On the *storage side* (§ 5), to fit on-device budgets while avoiding stalling NPU prefill, we design a multi-tier storage manager spanning the NPU buffer, CPU memory, and flash. It uses a hybrid on-disk data structure combining prefix trees with hash and semantic indices to efficiently locate both prefix and non-prefix reusable chunks. It also takes a lightweight, cost-aware prefetch and eviction policy that incorporates the differential cost of reloading versus recomputing prefix and non-prefix chunks, ensuring hot KV data stays close to the compute units. To prevent data movement from dominating latency (§ 6), we jointly schedule storage I/O and CPU-NPU execution based on parallelizability analysis across model shards and input chunks. Together, these components form a cross-layer co-design with mutually complementary compute and storage decisions, as validated by our ablation study.

We summarize the key contributions as follows:

- We identify fundamental system bottlenecks in on-device LLM serving with KV reuse: the architectural mismatch between dynamic KV recomputation and the static compiled graphs required by mobile NPUs, exacerbated by heavy KV cache transfer under tight on-device memory bandwidth and capacity.

- We develop an intra-graph construction and a general multi-graph scheduling abstraction for dynamic inputs over pre-compiled static graphs, along with a cross-layer compute-storage design that jointly optimizes KV recomputation, placement, and movement.
- We take 3 different workloads, 4 LLMs from Qwen3 and Llama3.2 series, and 3 smartphones for evaluation. Evaluation results reveal that our design delivers 1.5×–5.4× prefill speedup over original ExecuTorch [12] with no reuse and 1.3×–2.5× over prefix caching, cuts TTFT by 40%–60%, and avoids the roughly 30% quality drop of direct full non-prefix reuse.

2 Background

2.1 KV Cache Prefix and Non-Prefix Reuse

For transformer-based LLMs, the prefill phase computes per-layer Key-Value (KV) tensors for the input tokens. These tensors form the KV cache, which can be reused when the same context reappears, thereby reducing redundant prefill computation and reducing TTFT.

For input prompt, tokens can be categorized into 3 types: *prefix reusable*, *non-prefix reusable*, and *new* tokens, as exemplified in Figure 3. **Lossless prefix caching** strategy directly reuses the KV cache of a prompt prefix when a new request shares the same leading tokens as some previously cached prompts (① in Figure 3). Because the KV states of a prefix do not depend on subsequent tokens, this reuse is lossless and preserves generation quality, therefore widely adopted in modern cloud-based LLM serving systems, such as VLLM [22], SGLANG [53], and ChunkAttention [51], but its benefit is limited to prefix reusable texts that posit at the beginning of the prompt.

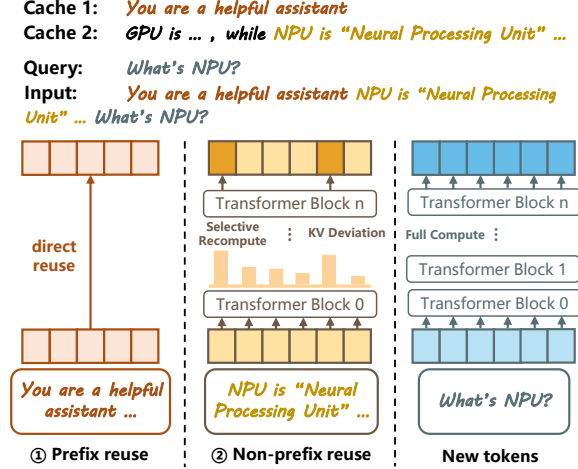


Figure 3. Illustration of 3 types of tokens in a prompt: prefix, non-prefix, and new tokens. ① Prefix caching directly reuses KV states for shared prefixes. ② Non-prefix KV chunks recompute only high KV deviated tokens and reuse the rest.

A more flexible but **lossy non-prefix chunk reuse** strategy targets reusable non-prefix chunks in the prompts, offering a substantially larger reusing space. Direct reusing non-prefix chunks in Prompt Cache [15] may significantly lose accuracy because they cannot recover the true positional dependency and cross-chunk attention of non-prefix chunks. Instead, CacheBlend [50], which recomputes a small set of selected tokens with high KV deviation while reusing the remaining chunk-level KV cache (② in Figure 3), provides a practical middle ground between exact but restrictive prefix reuse and expensive full recomputation for non-prefix reuse. Based on the insight that KV deviation is strongly correlated across layers [50], the algorithm compares cached KV with fully recomputed KV in the first few layers, selects the top- K most deviated tokens, and selectively recomputes them in the subsequent layers. Thus, it reduces the computational FLOPs to approximately the recompute ratio r of full computation and induces only tiny precision loss.

As illustrated in Figure 4(a), the selective non-prefix KV recompute workflow of CacheBlend algorithm features 5 forms of dynamicity in its computational graph. ① *Input shape dynamicity*, where prompt lengths vary significantly across requests. ② *Recomputation selection dynamicity*, where the subset of tokens to recompute is determined dynamically according to KV deviation. ③ *Dynamic numerical range*, where the KV comparison operation is highly precision-sensitive. Intermediate summed squared error of i -th token of the layer 0 is computed as:

$$\Delta KV[i] = \left\| KV_{\text{layer } 0}[i] - \widehat{KV}_{\text{layer } 0}[i] \right\|_F^2, \quad (1)$$

which spans a wide numerical range from 0 to 2000 and exhibits substantially different distributions across prompts as

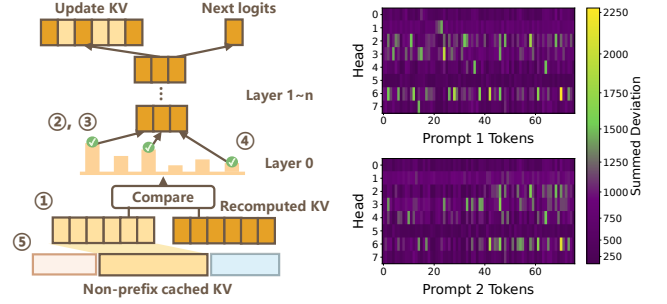


Figure 4. Characteristics of non-prefix KV reuse and selective recompute algorithm.

Figure 4. Characteristics of non-prefix KV reuse and selective recompute algorithm.

shown in the KV deviation heatmap in Figure 4(b). Such large dynamic ranges make quantization difficult and can lead to overflow or underflow when statically quantized to 8/16-bit integers on mobile NPUs. ④ *Output last token dynamicity*, where the final prompt token must always be selected to produce logits for subsequent autoregressive decoding. ⑤ *Reuse-pattern dynamicity*, where the token reuse pattern within a prompt is determined at runtime, with prefix-reusable tokens, non-prefix-reusable tokens, and new tokens appearing in arbitrary interleaved combinations and lengths.

2.2 Mobile Hardware Characteristics

Mobile NPU Characteristics. Mobile NPUs equipped with specialized matrix units can substantially accelerate long-context LLM prefill, often achieving 2–5× speedup over mobile CPU and GPU [47]. These matrix units process fixed-shape tensor tiles streamingly, which strongly favor **static graphs**, i.e., offline pre-compiled computational graphs with statically determined operator shapes and workflow. In practice, NPU graph preparation involves graph construction, optimization, and compilation, whose overhead is only acceptable when performed offline. Unlike cloud GPUs, which leverage symmetric tensor cores and SIMT parallelism for flexible programmability and irregular-shape support, mobile NPUs deliberately trade this flexibility for efficiency: regular tensor shapes enable efficient mapping onto matrix units and streamlined hardware scheduling under stringent power and thermal budgets, which is what underlies their performance-per-watt advantage. Consequently, they require specialized techniques to convert dynamic execution patterns into static graphs. This execution preference is not vendor-specific, but is shared across major mobile NPU platforms including Qualcomm HTP [20], MediaTek APU [19], and Apple ANE [18], and it persists across the three generations of Hexagon NPUs evaluated in this work.

In addition, mobile NPUs achieve the highest compute and memory efficiency on **integer GEMM** workloads, such as

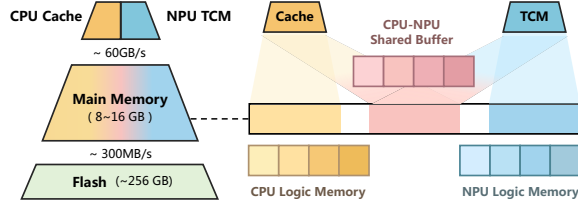


Figure 5. CPU-NPU memory hierarchy of mobile devices.

$W_{INT4AUINT16}$ [20, 44], while floating-point execution is comparatively more expensive. Mobile NPUs also favor **static quantization**, where quantization parameters are calibrated offline and reused during inference. Although effective for regular LLM prefill and decode, static quantization is less suitable for selective recomputation, because the compare-and-select stage relies on precision-sensitive KV difference computation whose distributions are highly input-dependent and difficult to calibrate offline.

On-Device Memory Hierarchy. The on-device memory hierarchy typically follows a three-tier structure, as illustrated in Figure 5: (1) CPU cache and NPU tightly coupled memory (TCM) SRAM at the top; (2) CPU-NPU shared DRAM in the middle; and (3) flash storage at the bottom. While the CPU cache and NPU TCM offer low latency for compute-intensive operations, their capacity is strictly limited (often < 10 MB). On modern SoCs like the Qualcomm HTP [20], the DRAM layer is physically shared between the CPU and NPU, while logically divided into CPU logic memory, NPU logic memory, and a CPU-NPU shared buffer. At the lowest level, flash storage serves as persistent storage for models and KV data.

Table 1. Comparison between on-device and cloud memory hierarchies for LLM serving and KV storage.

Platform	Memory Level	Capacity	Bandwidth	Interconnect
Device	Cache / TCM	2–8 MB	–	–
	CPU LPDDR	8–16 GB	~60 GB/s	On-chip Bus
	Flash	128 GB–1 TB	~500 MB/s	I/O Bus
Cloud	GPU Cache	~100 MB	–	–
	GPU HBM	24–640 GB	1–3 TB/s	NVLink
	CPU DRAM	0.5–10 TB	50–300 GB/s	PCIe
	Distributed Storage	10–1000 TB	0.5–2 GB/s	RDMA

In contrast to cloud-based LLM serving, on-device platforms operate under substantially tighter memory-capacity and bandwidth constraints. Cloud deployments typically organize memory hierarchically across GPU device memory, CPU host memory, and distributed storage, interconnected through high-bandwidth fabrics such as NVLink, PCIe, and RDMA. Table 1 summarizes cloud-based and on-device memory architectural disparities, highlighting the bandwidth wall faced by mobile-class hardware.

3 New Key Challenges

Given the algorithmic characteristics of prefix and non-prefix KV reuse (§ 2.1) and the mobile hardware characteristics (§ 2.2), two key challenges emerge from the compute and storage perspectives.

3.1 Recomputation Dynamicity vs. NPU Staticity

From the compute perspective, the key challenge lies in the mismatch between the highly dynamic execution patterns introduced by selective recomputation for non-prefix reusable tokens and the pre-compiled static execution graphs optimized by mobile NPUs.

The 5 forms of dynamicity discussed in § 2.1 translate into several concrete conflicts with mobile NPU execution. Specifically, dynamic tensor shapes (①) conflict with statically fixed tensor sizes in compiled graphs; dynamic computational flows (② and ④) conflict with static operator graphs; dynamic numerical ranges (③) conflict with static quantization parameters; and dynamic reuse patterns (⑤) conflict with static graph invocation strategies. These conflicts jointly complicate the design of efficient intra-graph construction and quantization schemes, as well as inter-graph scheduling and invocation optimization.

3.2 Large KV Size vs. Limited Bandwidth and Space

From the storage perspective, the key challenge arises from the mismatch between the large size of KV tensors and the limited bandwidth and capacity of on-device memory hierarchies, as KV chunks must be loaded from lower memory levels with limited bandwidth and stored under tight memory capacity constraints, making efficient hierarchical KV management difficult.

Due to constrained I/O bandwidth, loading large KV tensors from flash storage can introduce substantial latency. For example, the 8-bit quantized KV cache of Qwen3-1.7B occupies approximately 56 KB per token, corresponding to 56 MB for a 1024-token prompt. On Xiaomi 15 Pro, loading and mapping such a prompt from flash storage into the memory pool takes approximately 200 ms, which can diminish the execution advantage of NPUs by causing them to wait if the latency is not carefully hidden.

Limited on-device memory and storage capacity prevent KV tensors from being retained indefinitely. As a result, eviction and compaction mechanisms are required to periodically or adaptively reclaim and reorganize storage space. These maintenance operations must also be carefully overlapped with execution; otherwise, excessive secondary-storage I/O and fragmentation can significantly degrade performance and expose additional latency to users.

The above new challenges make existing cloud-based KV reuse approaches inapplicable, demanding new compute-storage co-designs for efficient on-device KV reuse.

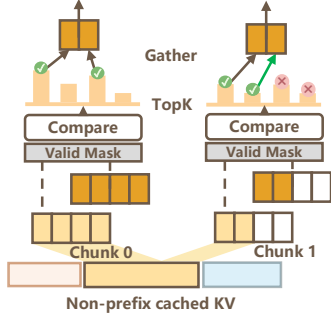


Figure 6. NPU static KV recompute graph on layer 0.

4 NPU Recompute for Non-Prefix Reuse

To resolve the conflicts between the dynamicity of non-prefix reuse selective recompute algorithm and the staticity of NPU pre-compiled workflows, we propose intra-graph design that converts dynamic algorithm to static NPU computation graph, searches for the latency-optimal static tensor shape to optimize the graph, and supports precision-sensitive operators with mixed-precision operators to cover dynamic numerical range of intermediate tensors. We further propose inter-graph dynamic programming to optimize dynamic reuse patterns via merging chunks and reducing padding.

4.1 Intra-Graph Design for Dynamic Recomputation

NPU Static Graph Construction. Converting the non-prefix reuse algorithm CacheBlend to static NPU graph requires addressing the dynamicity detailed in §2.1. The resulting static NPU KV recompute graph is illustrated in Figure 6. The dynamic input shape (dynamicity ①) is addressed by chunking the non-prefix tokens into static size slices, padded if not divisible by the chunk length. A valid mask is applied before KV deviation computation to exclude padded tokens from the Top-K selection process, preventing invalid tokens from being mistakenly selected for recomputation. Additionally, the mask guarantees that the final token is always selected (green arrow in Figure 6) wherever its position in last chunk is (dynamicity ④), ensuring correct generation of the final logits. This *chunk-pad-mask* adapts the static graph to dynamic input and output shapes.

After Top-K token selection, the *scatter-gather* mechanism gathers the recomputed KV of tokens with high KV deviation and scatters them back into the reused KV cache, replacing the corresponding reused KV entries to preserve generation quality, which resolves the dynamicity in token selection and KV update workflow (dynamicity ②).

Latency-Optimized Graph Size Configuration. The static graph input size is critical to improve NPU graph utilization and execution efficiency, thus maximizing hardware utilization, such as NPU TCM capacity and matrix units (e.g.,

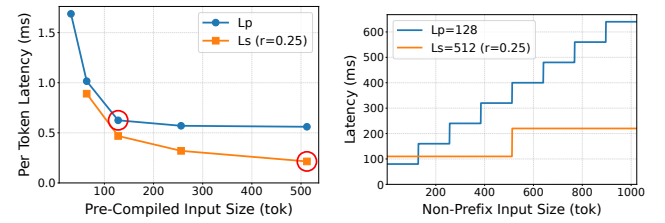
Qualcomm QNN HMX tiles), while minimizing padding overhead caused by indivisible input shapes. An optimal graph size can be determined by minimizing the expected latency over a representative target workload. In our design, the pre-fill graph size L_P and the selective recomputation graph size L_S are optimized. Given the latency of one graph invocation, denoted by $\text{Latency}_P(L_P)$ and $\text{Latency}_S(L_S)$, the latency for processing tokens of length n can be expressed as:

$$\begin{aligned} \text{Latency}_P(n) &= \lceil \frac{n}{L_P} \rceil \cdot \text{Latency}_P(L_P), \\ \text{Latency}_S(n) &= \lceil \frac{n}{L_S} \rceil \cdot \text{Latency}_S(L_S). \end{aligned} \quad (2)$$

The expected latency of graph G can be computed as:

$$\mathbb{E}(\text{Latency}_G) = \sum_{n=1}^N \text{freq}(n) \cdot \text{Latency}_G(n), \quad G \in \{P, S\}. \quad (3)$$

$\text{freq}(n)$ denotes the occurrence frequency of token length n collected from a representative workload.



(a) Graph execution per-token latency under different graph sizes. (b) Non-prefix tokens prefill latency under different input sizes.

Figure 7. Latency analysis of static graph input sizes.

Aligned with Qualcomm HMX tile utilization strategies [17], we restrict graph sizes to multiples of 32 and select the configuration that yields the lowest expected latency. As shown in Figure 7(a), per-token latency is initially high for small input lengths due to poor hardware utilization and then sharply decreases as input size grows and finally slowly converges. 2 red-circled latency-optimal points given by our optimization both correspond to the turning point of the per-token latency curve, beyond which increasing the graph input size no longer improves hardware utilization and instead introduces additional padding overhead. Figure 7(b) further illustrates the stair-step pattern relationship between latency and input size due to padding. It also shows that, under the latency-optimal configuration, selective recomputation (orange curve, $L_S = 512$) achieves 2–3× lower latency than ordinary prefill (blue curve, $L_P = 128$) for processing non-prefix reusable tokens.

Mixed-Precision Quantization for KV Deviation Computation. The KV deviation computation introduced in the selective KV recompute pipeline is precision-sensitive and input-dependent (dynamicity ③ analyzed in Section 2.1). Thus, directly quantizing it to INT8/UINT16 like other intermediate activations can lead to significant accuracy degradation. To address this, we propose a mixed-precision selective

KV recompute graph, where the deviation computation and Top-K comparison are all conducted on FP16 precision. We further apply a scale factor $\frac{1}{\sqrt{D_{head}}}$ to scale down the intermediate deviation tensor to prevent FP16 overflow. Such mixed-precision quantization successfully preserves the model precision, verified in the ablation study.

4.2 Inter-Graph Design for Dynamic Reuse Pattern

Within an input prompt, prefix reusable, non-prefix reusable, and new tokens may come interleavably (dynamicity ⑤). They invoke distinct graph calls by default: prefix reuse tokens incur no graph calls, non-prefix reusable tokens are processed by the selective recomputation graph, and new tokens are processed by the full prefill graph. A naive schedule which invokes static graphs solely according to token types may cause fragmentation between tokens of different types, often producing underfilled graph calls and wastes computation due to padding. To address this inefficiency under dynamic patterns, we propose a chunk merging algorithm based on dynamic programming to improve the static graph utilization. The key observation is that *non-cached new tokens can be safely processed by the selective recomputation graph*, since they are always selected for computation. Our algorithm can dynamically merge new tokens into adjacent under-utilized selective recomputation graphs to improve efficiency.

Let $T[i]$ be the minimum latency to process tokens up to position i . For the last chunk ending at i , we consider 2 choices: using the selective recomputation graph or using the prefill graph. This yields the optimal substructure transition of dynamic programming:

$$T[i] = \min\{T[i - l_s(i)] + t_s, T[i - l_p(i)] + t_p\}, \quad (4)$$

where t_s and t_p are the per-call latency of the selective recomputation and prefill graphs, respectively. Here, l_s is the maximum suffix length ending at i that can be absorbed by one selective recomputation graph call under graph capacity and recomputation-ratio constraint (i.e., at least r non-prefix tokens can be recomputed). l_p is the maximum suffix length ending at i that can be processed by one prefill graph call under its capacity. Intuitively, l_s may include both non-prefix reuse tokens and a bounded number of adjacent new tokens, whereas l_p simply packs as many trailing tokens as the prefill graph allows. In Equation 4, the first term $T[i - l_s(i)] + t_s$ is the optimum of the subproblem if the recomputation graph is used, while the second term $T[i - l_p(i)] + t_p$ corresponds to using the prefill graph. The dynamic program therefore globally chooses whether each trailing chunk should be executed by recomputation or prefill, rather than making a purely local greedy decision. The formal formulation of this constrained graph scheduling and complete pseudo code are provided in the supplementary material.

Figure 8 shows a toy example: when the recomputation graph processes up to 4 tokens, the prefill graph processes

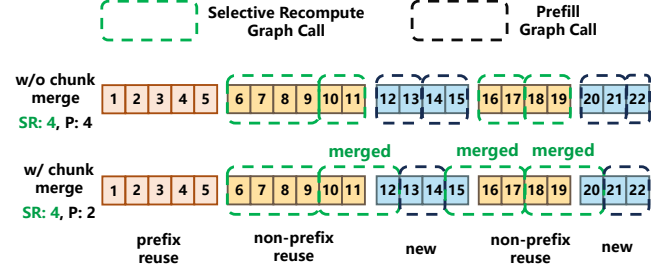


Figure 8. Chunk merging algorithm example, which saves 2 prefill graph calls and reduces $\approx 25\%$ latency.

up to 2 tokens, and $r = 50\%$, new tokens 12, 15, and 20 can be merged into adjacent recomputation calls. As a result, the number of prefill graph invocations is reduced from 4 to 2, yielding an approximate 25% latency reduction by utilizing otherwise underfilled recomputation graphs.

4.3 Generality of Our Design

Our compute-side design is not specific to Qualcomm HTP. It is applicable to mobile accelerators that expose ahead-of-time compiled static graphs over fixed-shape tensors, such as MediaTek APU and Apple ANE. Under this execution model, the intra-graph mechanisms in §4.1, including chunk-pad-mask and scatter-gather, can be expressed using standard tensor operators supported by these backends. The inter-graph scheduler in §4.2 depends only on the capacity and measured latency of the available pre-compiled graphs, and is therefore independent of a particular accelerator implementation. Porting to a new backend requires recalibrating the graph sizes L_p and L_s and the hardware-specific alignment granularity following the procedure in §4.1. Thus, we expect migration to require backend integration and profiling rather than redesigning the compute-side mechanisms.

5 On-Device Hierarchical KV Storage

To stably supply reusable KV tensors to NPU under the strict on-device memory bandwidth and capacity constraints, we design a hierarchical KV storage system upon the on-device memory hierarchy. Inside, the data structures in memory and flash storage leverage heterogeneous memory characteristics to balance reuse efficiency and data movement overhead. Prefetch and eviction policies are also activated to mitigate cache misses across memory and storage tiers and consequently reduce end-to-end latency. The architectural design of KV caching system is illustrated in Figure 9. The system majorly consists of 3 structures: NPU KV manager, CPU KV memory pool, and the flash KV DB. During the workflow, KV tensors are efficiently matched and loaded from the flash DB and (pre)fetches to the CPU memory pool. When transmission signals of NPU are received, the tensors are further transferred to the CPU–NPU shared buffer for NPU to consume.

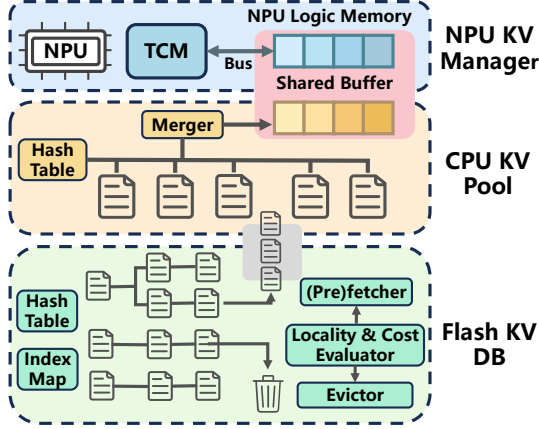


Figure 9. On-device hierarchical KV caching system.

5.1 NPU-CPU In-Memory KV Manager

NPU KV Manager. A fixed-size (context-length) KV buffer on CPU–NPU shared buffer, typically on the order of 10^1 MB is allocated by the NPU KV manager. This buffer contains the KV cache of the running session, where NPU directly loads KV tensors in this region with DMA into TCM and consumes it by the matrix unit. The manager controls its sharing behavior between CPU and NPU and updates the buffer throughout LLM executions.

CPU KV Memory Pool. A medium-sized KV pool is maintained in CPU memory, typically on the order of 10^2 MB, and is implemented as a lightweight hash table based structure that indexes cached KV tensors using their corresponding underlying database row IDs. To save memory, the pool stores KV chunks that are either newly generated by the NPU or (pre)fetcher from flash storage, rather than being a full tree-based structure as cloud-based existing works [21, 53]. Upon receiving a new input request, the system first queries the CPU KV pool before accessing flash storage to enable low-latency reuse. Eviction is triggered under system memory pressure, such as when available memory becomes limited due to contention from other mobile applications.

5.2 Prefix and Non-Prefix Hybrid Structure in Flash

We organize KV caches in on-device flash storage using a tree–hash–index hybrid structure implemented on top of SQLite [13], which jointly supports both prefix-based and non-prefix KV reuse while maintaining efficient lookup and moderate storage overhead. This storage layer is designed to operate at GB scale and can dynamically trigger eviction when requested by user or system under storage pressure.

Specifically, following prior work [21, 53], we construct a prefix tree to support efficient prefix reuse and hierarchical organization of cached KV entries. However, tree-based structures alone cannot efficiently support non-prefix reuse. To address this limitation, we augment the structure with

both hash-based and semantic indexing mechanisms. For non-prefix reuse, each cached KV chunk is associated with a compact 64-bit hash code, enabling fast lookup of identical reusable chunks. For subchunk-level reuse, we additionally maintain lightweight semantic indices, including sparse embedding based on extracted keywords and truncated dense embedding vectors. Dense embedding is only in RAG use cases where embedding is already computed and stored in the workflow. This structure enables fast localization of potential non-prefix chunk candidates that may contain reusable subchunk sequences, even when the text is not identical at chunk level. Linear-time fast longest common substring (LCS) algorithm is further applied to identify and finalize the reusable subchunks among the retrieved candidates.

Together, the prefix tree for prefix reuse, and hash table and semantic index for non-prefix reuse, form a unified flash storage organization that supports both types of KV reuse across diverse long-context workloads. The detailed database schema and storage organization are described in the supplementary material.

5.3 Prefetch and Eviction

Asynchronous Prefetch. To improve prefix hit rate in the CPU memory pool, the system performs asynchronous prefetch between flash and main memory. During idle I/O periods, prefix-reusable chunks with high predicted reuse likelihood are proactively loaded into memory to reduce future access latency. To avoid unnecessary memory pollution, the prefetch policy selectively prioritizes chunks that are likely to participate in future prefix reuse or are strongly correlated with recently accessed chunks.

Cost-Aware Eviction. We jointly consider chunk reusability and eviction overhead to compute an overall eviction score for each chunk. Chunks that are both highly reusable and expensive to recompute or reload are preferentially retained under constrained memory and storage budgets, while lower-value chunks are evicted when necessary. This improves cache utilization and reduces end-to-end TTFT.

For chunks in the CPU memory pool, eviction overhead mainly corresponds to reload latency from flash storage for prefix-reusable chunks, while the cost for non-prefix-reusable chunks is negligible because their loading latency can largely overlap with execution (§ 6.1).

For chunks stored in flash, eviction may incur not only the recomputation cost of the evicted chunk itself, but also additional overhead from disrupting the prefix reuse chain. In particular, evicting a chunk on a prefix path can cause its descendant chunks to lose prefix reusability (Figure 20), thereby increasing future recomputation cost. Our eviction policy explicitly models this cascading effect when making retention decisions.

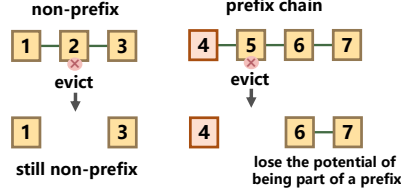


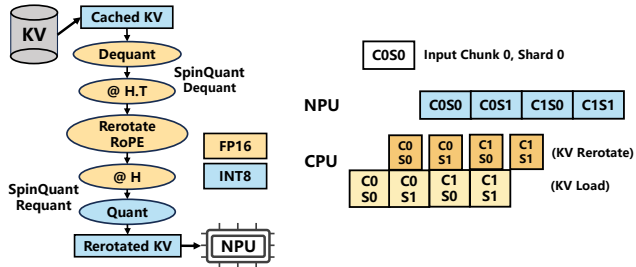
Figure 10. Evicting chunks on a prefix chain breaks future prefix reuse for subsequent chunks.

6 Compute–Storage Pipeline Overlap

Due to the limited bandwidth of on-device memory hierarchies, optimizing compute and storage independently is insufficient. Our system overlaps KV loading, rerotation, storing, and NPU execution through asynchronous pipeline parallelism to hide data-movement latency.

6.1 KV Loading Latency Hiding

During KV preparation, KV tensors are retrieved from storage and rerotated for non-prefix reuse before being consumed by the NPU.

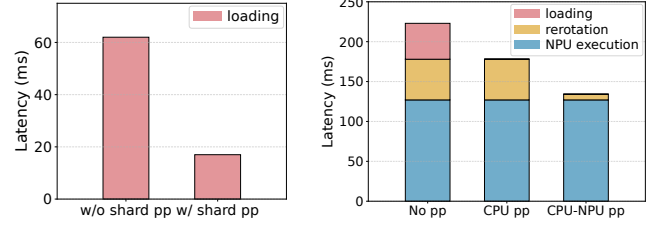


(a) CPU KV rerotation. (b) IO–CPU–NPU pipeline parallelism of KV preparation and LLM inference.

Figure 11. IO–CPU–NPU overlapping pipeline.

IO-CPU-NPU Overlapping. To enable non-prefix KV reuse, the rotary embedding (RoPE) of K tensors is rerotated according to their original positions in cached prompts and their new positions in incoming queries [15, 50, 51]. In the quantized on-device workflow, this process additionally involves KV dequantization and requantization with Hadamard transforms introduced by 8-bit SpinQuant [27]. Since rerotation is memory-intensive¹, it is assigned to the CPU and overlapped with compute-intensive NPU execution. As illustrated in Figure 11(a), both I/O-intensive KV loading and memory-intensive rerotation are executed asynchronously on the CPU to avoid stalling the NPU.

¹Fast Walsh–Hadamard transformation has $O(n \log n)$ complexity and is memory-intensive in our setting.



(a) Shard-level parallelism reduces prefix loading time 4×. (b) IO–CPU–NPU parallelism makes data preparation latency invisible to NPU execution.

Figure 12. Hiding KV loading and rerotation latency via pipeline parallelism. (Qwen3-1.7B on Xiaomi 15 Pro)

Chunk-Shard Two-Dimensional Pipeline Parallelism.

Mobile NPUs execute LLMs in multiple model shards while prompts are processed in chunks [47], enabling pipeline parallelism along both dimensions. As illustrated in Figure 11(b), while the NPU executes shards of Chunk 0, the CPU concurrently prepares KV tensors for upcoming shards of Chunk 0 and subsequent Chunk 1. Figure 12 shows that incorporating shard-level parallelism reduces the non-overlapped loading latency of prefix chunks and the first non-prefix chunk by 4×. Overall, this two-dimensional pipeline substantially increases overlap opportunities and effectively hides KV loading and rerotation latency.

6.2 KV Storing Latency Hiding

Synchronously serializing KV tensors from the CPU memory pool to flash storage can introduce substantial user-visible latency due to limited secondary-storage bandwidth. To mitigate this overhead, we adopt a lazy and asynchronous KV persistence mechanism.

Instead of synchronously writing KV tensors to flash immediately after generation, newly produced KV chunks are first retained in the CPU memory pool and marked as dirty entries, while the actual serialization and storage operations are deferred and executed asynchronously by background threads when the I/O bus is idle. For example, KV storing can be overlapped with decoding. In addition, write operations are batched whenever possible to reduce flash I/O amplification and metadata overhead. This design effectively hides most flash write latency from the critical inference path.

7 Evaluation

7.1 Experimental Setup

Use Cases and Datasets. We take the following three representative on-device LLM use cases.

- For *document QA*, we randomly sample 400 easy-level queries from HotpotQA [49]. For each query, we split its associated context into 1024-character chunks to

build the context chunks document database, and retrieve the top-6 chunks based on the L2 distance between embeddings generated by Qwen3-Embedding-0.6B [52].

- For *long chat history QA*, we use LoCoMo-MC10 [34], derived from LoCoMo [29]. Two sessions of very long chat history with 74 single-hop QA questions are selected to evaluate reuse under long conversation contexts. Top-6 related dialog chunks are retrieved and combined into the prompt, with the same setting as HotpotQA.
- For *agent skill use*, we construct a skill-oriented evaluation set from SkillsBench [6], including 10 shared skills and 39 associated task instructions. Each testing sample contains a reusable skill description and a task instruction, enabling evaluation under shared skill contexts.

These workloads cover both contiguous prefix reuse and more general non-prefix reuse patterns, where reusable content may appear as retrieved document chunks, historical conversation segments, or shared skill descriptions.

Devices. Our evaluation includes one mid-range device, Meizu 21, and two high-end devices, Xiaomi 15 Pro and Honor Magic 8. Their hardware specifications are summarized in Table 2. All experiments are conducted using the Qualcomm Hexagon NPUs available on these devices.

Table 2. Mobile devices evaluated in our experiments.

Device	SoC	Mem	CPU	GPU	NPU
Meizu 21	Snapdragon 8 Gen 3	12GB +256GB	1*Cortex-X4+5*A720 +2*A520	Adreno 750	Hexagon V75
Xiaomi 15 Pro	Snapdragon 8 Elite	16GB +512GB	8*Oryon	Adreno 830	Hexagon V79
Honor Magic 8	Snapdragon 8 Elite Gen 5	12GB +512GB	8*Oryon	Adreno 840	Hexagon V81

Models. We take Qwen3-1.7B and Qwen3-4B [39], and Llama3.2-1B-Instruct and Llama3.2-3B-Instruct [37, 38]. Models are majorly quantized into Int4.

Base Engine and NPU Backend. We adopt ExecuTorch [12] as the base mobile LLM engine, which provides state-of-the-art performance on NPUs together with a stable and extensible deployment framework for on-device LLM inference. We target Qualcomm HTP NPUs, motivated by their broad deployment and mature public software stack.

Baselines. We compare against three baselines representing different KV reuse strategies:

- *no reuse*: the original ExecuTorch [12] implementation without KV reuse.
- *prefix caching*: an implementation with tree-based prefix caching, reproducing SGLANG Radix-Tree prefix reuse on device [53].

- *full reuse*: an implementation that directly reuses both prefix and non-prefix KV chunks without selective recomputation, reproducing Prompt Cache full reuse on device [15].

The KV storage backend used by the *prefix caching* and *full reuse* baselines is also implemented by us on top of SQLite to ensure a fair comparison under the same on-device storage stack. We do not directly compare against cloud-based systems, such as vLLM, SGLang, LMCache, or RAGCache, because they are designed for GPU server environments and do not support mobile hardware.

Metrics. For all three use cases, we report the average TTFT (ms) over representative datasets to evaluate system efficiency. In addition, to validate the effectiveness of our CacheBlend style selective recomputation algorithm for non-prefix reuse on mobile devices, we evaluate generation quality on the document QA and chat-history QA workloads and report the corresponding accuracy. We also report the average system power and total energy consumption during prefill, measured via Android BatteryManager, to evaluate whether latency reductions translate into energy savings.

Key Configurations. The graph sizes of the NPU selective recomputation graph and prefill graph are set to the latency-optimal values identified in § 4.1: $L_S = 512$ and $L_P = 128$. To preserve generation quality while fully utilizing the HMX GEMM tiles [17], the recomputation ratio is set to $r = 0.25$, ensuring that intermediate tensor shapes remain multiples of 32.

7.2 End-to-End Improvements

Reduced TTFT with Negligible Quality Degradation.

Figure 14 illustrates the quality–latency tradeoff on HotpotQA and LoCoMo using Qwen3-1.7B and Qwen3-4B on Xiaomi 15 Pro. The ideal operating region lies in the upper-left corner, corresponding to lower TTFT and higher task accuracy.

Compared with *no reuse* and *prefix caching*, our system substantially shifts the operating point toward this region, reducing TTFT by approximately 40%–60% with only $\leq 4\%$ accuracy degradation. Although *prefix caching* preserves full precision, its latency improvement is limited because it only exploits reusable contiguous prefixes.

In contrast, the more aggressive *full reuse* achieves slightly lower TTFT but incurs significant quality degradation (approximately 30%) due to direct non-prefix KV reuse without selective recomputation. Moreover, *full reuse* is only 12% faster than our approach because its rerotation overhead cannot be effectively hidden through pipeline overlap and becomes exposed on the critical path. Therefore, we exclude *full reuse* from the subsequent evaluations.

Efficiency Evaluation across Tasks, Models, and Devices. Figure 13 compares TTFT among *no reuse*, *prefix caching*,

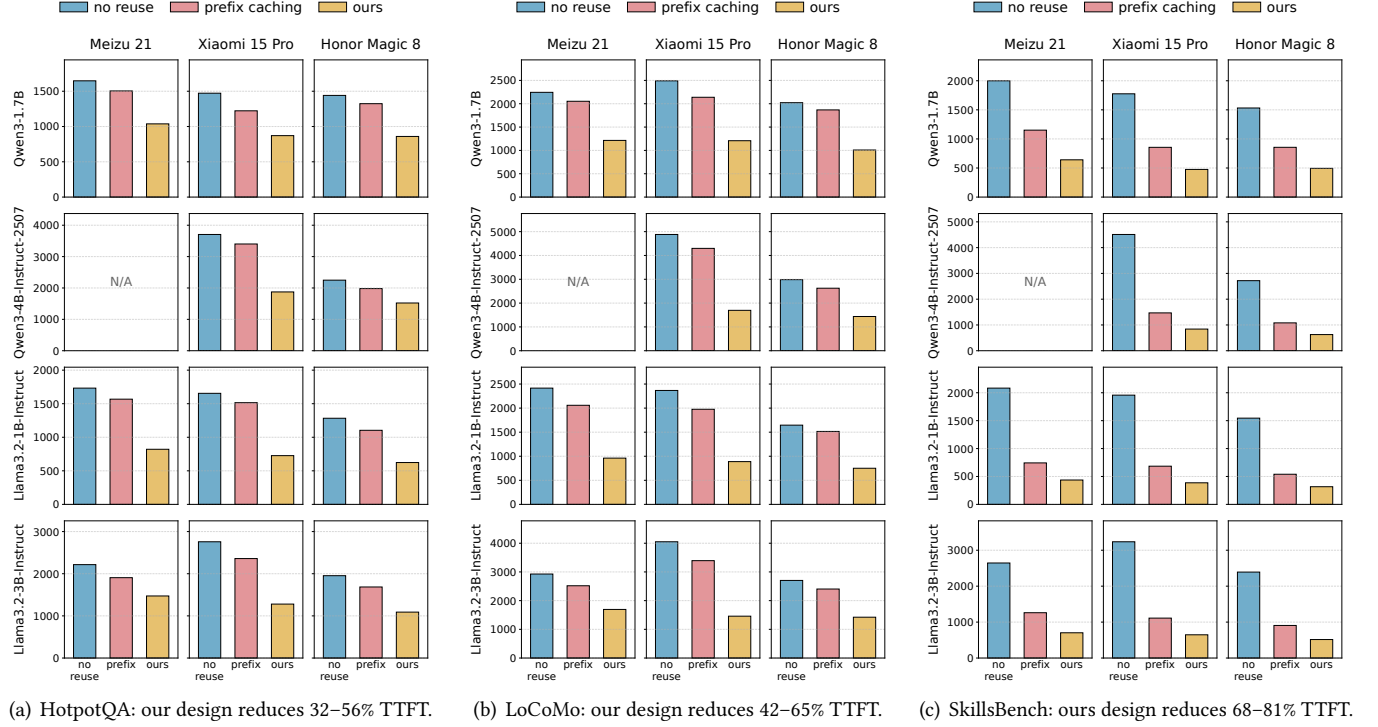


Figure 13. End-to-end TTFT across 3 workloads, 4 LLMs, and 3 smartphones. Our system consistently reduces TTFT compared with no reuse (original ExecuTorch) and prefix caching. The TTFT reduction ranges in each subcaption are measured against original ExecuTorch across all available model-device pairs.

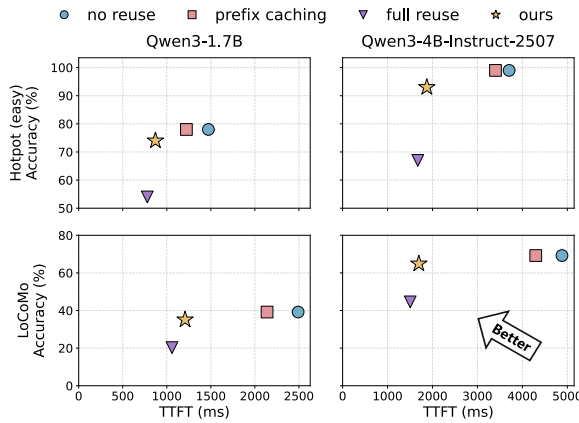


Figure 14. Quality-latency tradeoff on HotpotQA and LoCoMo using Qwen3-1.7B and Qwen3-4B on Xiaomi 15 Pro. Our system (yellow star) reduces TTFT substantially while preserving generation quality.

and *ours* across different workloads, LLMs, and smartphones. Across all configurations, our system consistently achieves the lowest TTFT, delivering $1.5\times$ – $5.4\times$ prefill speedup over *no reuse* and $1.3\times$ – $2.5\times$ speedup over *prefix caching*.

In the agent skill-use workload, reuse patterns are largely prefix-dominated because multiple tasks share the same skill

descriptions and tool instructions. As a result, both *prefix caching* and our system reduce TTFT by more than 50%. Our system further improves over *prefix caching* through optimized prefetching and compute-storage pipeline overlap.

In contrast, document QA and long-chat workloads exhibit substantially more non-prefix reuse. Retrieved document chunks or dialog histories are dynamically assembled into prompts with different orders and combinations, making reusable prefixes cache hits much more difficult. Under such patterns, *prefix caching* achieves only limited gains ($1.1\times$ – $1.2\times$), especially on mobile devices where constrained memory capacity causes frequent Radix-Tree eviction and prevents caching all prompt combinations. In contrast, by supporting both prefix and non-prefix KV reuse through selective recomputation, our system continues to achieve substantial TTFT reduction, providing $1.5\times$ – $2.9\times$ speedup in these workloads.

Energy Reduction. We measure prefill energy consumption on Xiaomi 15 Pro with Qwen3-4B-Instruct-2507 across the three datasets and compare our system against *no reuse*. Our system maintains a comparable average power draw to *no reuse*: 6.34 W vs. 6.68 W, indicating that the additional KV management operations do not materially increase system power. Consequently, the reduction in prefill latency directly

translates into lower energy consumption. As shown in Table 3, our system reduces prefill energy by 52% on HotpotQA, 67% on LoCoMo, and 77% on SkillsBench. These reductions closely track the corresponding TTFT improvements in §7.2, confirming that our system substantially reduces both latency and energy consumption.

Table 3. Prefill energy reduction measured on Xiaomi 15 Pro with Qwen3-4B-Instruct-2507 across 3 datasets.

Energy (J)	HotpotQA	LoCoMo	SkillsBench
No Reuse	24.76	32.61	18.02
Ours	11.89	10.75	4.13
	-52.0%	-67.0%	-77.1%

7.3 Sensitivity Analysis

For a better understanding of on-device KV reuse mechanism, we analyze how the reuse ratio and prompt length affect the end-to-end prefill latency. We report the latency reduction range across all available model-device pairs.

Varying Prefix Reuse Ratio. Figure 15 shows that prefix reuse consistently reduces prefill latency as the reuse ratio increases, and the benefit becomes more significant for longer prompts. At 3072 tokens, the full-prefill baseline takes 2.15–5.93 s across different models and devices. With 75% prefix reuse, our system reduces the prefill latency by 1.54–4.43 s, corresponding to a 67.4–78.3% reduction. When the reuse ratio reaches 100%, the latency reduction improves to 94.1–99.7%, leaving 9–215 ms residual prefill latency. These results indicate that when reusable KV cache forms a contiguous prefix, our system can reuse with little extra overhead, while the speedup is more obvious for higher hit ratio.

Varying Non-Prefix Reuse Ratio. Figure 16 shows that non-prefix reuse also reduces end-to-end prefill latency, although the residual latency is higher than prefix reuse. At 3072 tokens, 100% non-prefix reuse reduces the prefill latency by 1.08–3.80 s, corresponding to a 44.9–67.9% reduction over the no-reuse baseline. The remaining latency, 0.87–1.99 s, is due to non-prefix reuse and selective recompute overhead. Nevertheless, the end-to-end prefill latency remains substantially lower than full prefill, showing that the saved model computation outweighs the extra reuse-management overhead.

Overall, the results demonstrate that our method is consistently effective across different mobile devices and LLMs. Higher reuse ratios lead to larger latency reductions, and longer prompts further amplify the benefit of KV reuse. Prefix reuse provides the largest improvement due to its contiguous-cache structure, while non-prefix reuse offers a more general reuse capability with moderate additional overhead.

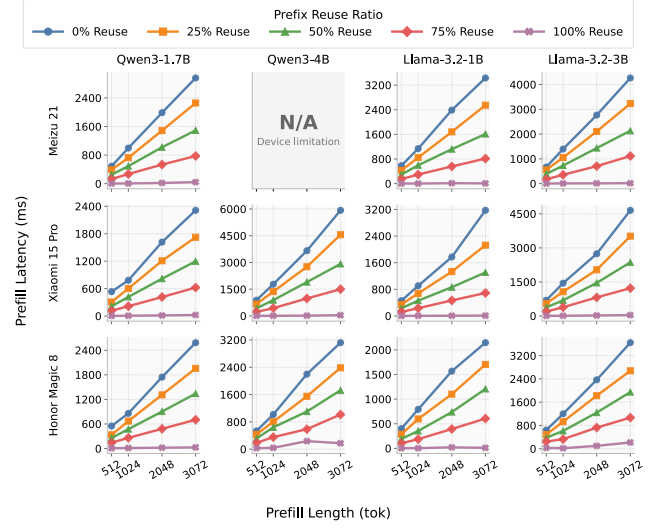


Figure 15. TTFT under different prefix reuse ratios and input lengths across 3 mobile devices and 4 models.

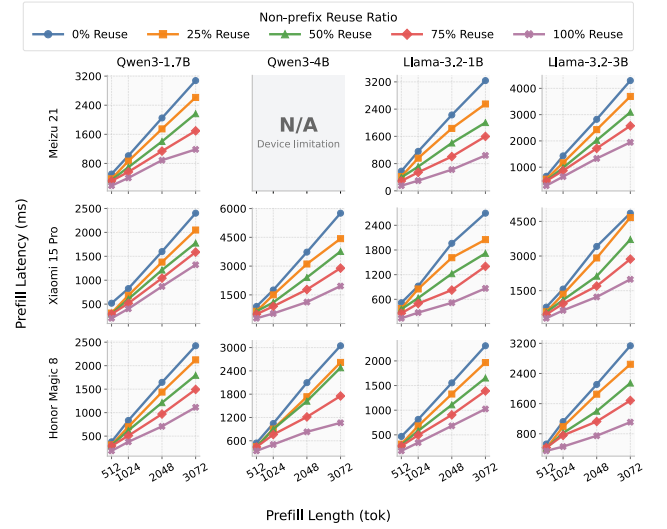


Figure 16. TTFT under different non-prefix reuse ratios and input lengths across 3 mobile devices and 4 models.

7.4 Ablation Study

We decompose end-to-end latency into five components: prefix/non-prefix matching, eviction and storage, KV loading, KV rerotation, and NPU execution. Under the full system configuration, NPU execution dominates TTFT, accounting for more than 95% of total latency, indicating that the overhead introduced by KV reuse is largely hidden and amortized.

As is illustrated in Figure 17, removing individual system designs introduces noticeable overheads at different stages. In particular, disabling graph-size optimization and

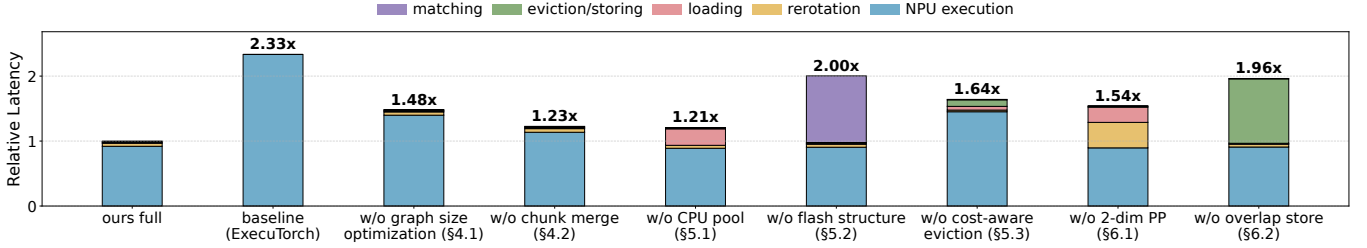


Figure 17. Ablation study and latency decomposition on Qwen3-1.7B, LoCoMo dataset, Xiaomi 15 Pro. Disabling individual components from our design increases TTFT by 1.21–2.33×.

the chunk-merge algorithm reduces graph utilization efficiency and increases overall latency by 48% and 23%, respectively. Removing the CPU memory pool increases KV loading latency from flash storage, adding approximately 20% latency. Replacing our hybrid structure in flash storage with a simple SQLite BLOB-based DB with naive structure increases matching overhead by an order of magnitude and nearly doubles overall latency. Disabling the eviction policy reduces the hit rate of non-prefix-reusable chunks by approximately threefold, resulting in a 60% latency increase. Without IO–CPU–NPU pipeline parallelism, KV loading and rerotation can no longer overlap with NPU execution, increasing latency to 1.5×. Similarly, disabling asynchronous KV storage exposes synchronous flash serialization overhead on the critical path and nearly doubles latency.

Overall, these results demonstrate that each major component contributes substantially to reducing end-to-end latency and improving KV reuse efficiency.

8 Related Works

Cloud-Based KV Reuse and Caching Systems. Cloud-based KV caching systems focus on organizing reusable KV caches across requests and cloud-based storage hierarchies. SGLANG [53] and RAGCache [21] employ prefix-aware tree structures, namely Radix Tree and knowledge tree, to store reusable prefixes. For non-prefix reuse, CacheBlend [50] introduces a distributed KV storage and retrieval mechanism. More generally, LMCache [26] provides a unified KV-cache layer for cache lookup, migration, and cross-engine sharing. Strata [46] studies hierarchical context caching for long-context serving, while Mooncake [36] advocates a KV-cache centric architecture that trades additional local storage for reduced recomputation. InfiniGen [23] further accelerates KV access through essential-KV-only prefetching.

On-Device LLM Acceleration. Existing approaches for accelerating on-device LLM inference primarily focus on optimized GEMM kernels, heterogeneous computing, quantized execution, and model sparsity.

ExecuTorch [12] provides high-performance NPU backends for Apple, MediaTek, and Qualcomm platforms through

extensible and stable interfaces. MNN [28, 42] improves mobile CPU and GPU inference through optimized KV-cache and weight layouts that enhance memory locality. MediaPipe [11] (built on LiteRT [10]) and MLC-LLM [30] (built on TVM [8]) primarily optimize GPU-based prefill, while mllm [47] accelerates with optimized NPU GEMM kernels. SmartMem [32] reduces memory overhead by eliminating unnecessary NC4HW4 GPU layout transformations self-attention and GEMM execution. NPU flash attention and mixed-precision GEMM optimization [17] are also explored in recent works. To better utilize memory bandwidth, HeteroLLM [7] introduces GPU–NPU co-execution parallelism. T-MAC [43] and T-MAN [44] leverage table-lookup based quantized kernels for efficient CPU and NPU inference. PowerInfer 2 [48], Apple Intelligence [3], and Neuralink [41] exploit activation sparsity for efficient weight offloading.

Despite these extensive optimization efforts, KV reuse remains unexplored in prior on-device LLM systems.

On-Device Tensor Storage Systems. SQLite [13], a widely used on-device database system, provides BLOB support for tensor storage but lacks tensor-specific optimizations. Existing systems such as MicroNN [35] and MobileRAG [33] focus on mobile vector databases, prioritizing embedding storage and building fast indices for vector approximate nearest neighbor (ANN) searches. Walle [28] and SFSL [31] provide updateable FlatBuffers-based [1] tensor-storage systems for fixed-sized model weights and embeddings, but inefficient for dynamic-sized KV tensors. For KV storage, SparKV [25] studies device–cloud KV transfer for mobile LLM inference, while MobiLoRA [24] introduces an on-device KV-cache design for multi-LoRA scenarios, but both consider intra-session runtime small-scale KV storage only.

However, persistent and dynamically updateable hierarchical KV storage architectures tailored for on-device KV reuse have not been studied in existing work.

9 Conclusion

In this work, we have designed and built an on-device KV reuse system to accelerate LLM inference on mobile NPUs. At the compute layer, we have transformed the dynamic selective recomputation workflow of non-prefix reuse into

efficient static graphs for mobile NPUs. At the storage layer, we have designed a hierarchical KV caching system with hybrid indexing structures for efficient matching and reuse of prefix and non-prefix KV tensors. Compute and storage are further pipelined and overlapped to hide system overheads. Our implementation on Qualcomm Hexagon NPUs demonstrates that efficient non-prefix KV reuse is practical on commodity smartphones. Across representative workloads, LLMs, and devices, our system consistently reduces TTFT over no reuse and prefix-only caching baselines, while maintaining negligible quality degradation. More broadly, this work provides a foundation for scalable long-context inference on resource-constrained mobile devices and open new opportunities for local-first personal intelligence that increasingly rely on persistent and reusable context.

Acknowledgments

We sincerely thank all the reviewers and our anonymous shepherd for instructive comments. This work was supported in part by China NSF grant (No. 62572299, No. 62441236, No. 62372296, No. 62432007, No. U24A20326, No. U25A6024, No. U25A20437), the Key Research and Development Program of Zhejiang Province (No. 2024C03270), Alibaba Innovation Research (AIR) Program (No. 56657574-1), CCF-Tencent Rhino-Bird Open Research Fund (No. RAGR20260126), and SJTU-Huawei Explore X Gift Fund.

References

- [1] Google 2025. *FlatBuffers: Memory Efficient Serialization Library*. Google. <https://github.com/google/flatbuffers>
- [2] Google 2026. *Gemini Introduces Personal Intelligence*. Google. <https://blog.google/innovation-and-ai/products/gemini-app/personal-intelligence/>
- [3] Keivan Alizadeh, Seyed Iman Mirzadeh, Dmitry Belenko, S. Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. 2024. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 12562–12584.
- [4] Apple. 2024. Introducing Apple's On-Device and Server Foundation Models. <https://machinelearning.apple.com/research/introducing-apple-foundation-models>.
- [5] Apple Inc. 2026. Apple Intelligence. <https://www.apple.com/apple-intelligence/>.
- [6] benchflow ai. 2026. SkillsBench: The first benchmark for evaluating how well AI agents use skills. <https://github.com/benchflow-ai/skillsbench>.
- [7] Le Chen, Dahu Feng, Erhu Feng, Yingrui Wang, Rong Zhao, Yubin Xia, Pinjie Xu, and Haibo Chen. 2025. Characterizing Mobile SoC for Accelerating Heterogeneous LLM Inference. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 359–374.
- [8] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: an automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation (OSDI '18)*. USENIX Association, Carlsbad, CA, USA, 579–594.
- [9] Hongchao Du, Shangyu Wu, Qiao Li, Riwei Pan, Jinheng Li, Youcheng Sun, and Chun Jason Xue. 2026. ClawMobile: Rethinking Smartphone-Native Agentic Systems. In *Proceedings of the Sixth European Workshop on Machine Learning and Systems (EuroMLSys '26)*. Association for Computing Machinery, 370–376.
- [10] Google AI Edge. 2025. LiteRT overview. <https://ai.google.dev/edge/litert>.
- [11] Google AI Edge. 2025. MediaPipe Solutions guide. <https://ai.google.dev/edge/mediapipe/solutions/guide>.
- [12] PyTorch Foundation. 2025. GitHub - pytorch/executorch: On-device AI across mobile, embedded and edge for PyTorch. <https://github.com/pytorch/executorch>.
- [13] Kevin P. Gaffney, Martin Prammer, Larry Brasfield, D. Richard Hipp, Dan Kennedy, and Jignesh M. Patel. 2022. SQLite: past, present, and future. *Proc. VLDB Endow.* 15, 12 (Aug. 2022), 3535–3547.
- [14] Georgi Gerganov. 2026. GitHub - ggml-org/llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>.
- [15] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems (MLSys '24, Vol. 6)*. 325–338.
- [16] Google. 2026. *Google AI Edge Gallery*. <https://play.google.com/store/apps/details?id=com.google.ai.edge.gallery>
- [17] Zixu Hao, Jianyu Wei, Tuowei Wang, Minxing Huang, Huiqiang Jiang, Shiqi Jiang, Ting Cao, and Ju Ren. 2026. Scaling LLM Test-Time Compute with Mobile NPU on Smartphones. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. Association for Computing Machinery, New York, NY, USA, 2157–2172.
- [18] Apple Inc. 2026. CoreML Documentation. <https://developer.apple.com/documentation/coreml>.
- [19] MediaTek Inc. 2026. NeuroPilot Documentation. <https://neuropilot-developer.mediatek.com/sphinx/neuropilot-8-public/html/>.
- [20] Qualcomm Technologies Inc. 2026. Qualcomm Documentation – HTTP Backend. https://docs.qualcomm.com/doc/80-63442-10/topic/http_backend.html.
- [21] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. 2025. RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation. *ACM Trans. Comput. Syst.* 44, 1, Article 2 (Nov. 2025), 27 pages.
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626.
- [23] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation (Santa Clara, CA, USA) (OSDI'24)*. USENIX Association, USA, Article 9, 18 pages.
- [24] Borui Li, Yitao Wang, Haoran Ma, Ligeng Chen, Jun Xiao, and Shuai Wang. 2025. MobiLoRA: Accelerating LoRA-based LLM Inference on Mobile Devices via Context-aware KV Cache Optimization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 23400–23410.
- [25] Hongyao Liu, Liuqun Zhai, Junyi Wang, Zhengru Fang, Jingshu Chen, and Jun Huang. 2026. SparkV: Overhead-Aware KV Cache Loading for Efficient On-Device LLM Inference. *IEEE Internet of Things Journal* 13, 14 (2026), 30016–30027.
- [26] Yuhao Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen

- Jiang. 2025. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665 [cs.LG] <https://arxiv.org/abs/2510.09665>
- [27] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. 2025. SpinQuant: LLM Quantization with Learned Rotations. In *Proceedings of The Thirteenth International Conference on Learning Representations (ICLR '25)*. 24 pages.
- [28] Chengfei Lv, Chaoyue Niu, Renjie Gu, Xiaotang Jiang, Zhaode Wang, Bin Liu, Ziqi Wu, Qiulin Yao, Congyu Huang, Panos Huang, Tao Huang, Hui Shu, Jinde Song, Bin Zou, Peng Lan, Guohuan Xu, Fei Wu, Shaojie Tang, Fan Wu, and Guihai Chen. 2022. Walle: An End-to-End, General-Purpose, and Large-Scale Production System for Device-Cloud Collaborative Machine Learning. In *Proceedings of USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX, Carlsbad, CA, USA, 249–265.
- [29] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Bangkok, Thailand, 13851–13870.
- [30] MLC team. 2023–2025. *MLC-LLM*. CMU Foundation and Language Model Center. <https://github.com/mlc-ai/mlc-llm>
- [31] Chaoyue Niu, Fan Wu, Shaojie Tang, Lifeng Hua, Rongfei Jia, Chengfei Lv, Zhihua Wu, and Guihai Chen. 2020. Billion-scale federated learning on mobile clients: a submodel design with tunable privacy. In *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (London, United Kingdom) (MobiCom '20)*. Association for Computing Machinery, New York, NY, USA, Article 31, 14 pages.
- [32] Wei Niu, Md Musfiqur Rahman Sanim, Zhihao Shu, Jiexiong Guan, Xipeng Shen, Miao Yin, Gagan Agrawal, and Bin Ren. 2024. SmartMem: Layout Transformation Elimination and Adaptation for Efficient DNN Execution on Mobile. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 916–931.
- [33] Taehwan Park, Geonho Lee, and Min-Soo Kim. 2025. MobileRAG: A Fast, Memory-Efficient, and Energy-Efficient Method for On-Device RAG. arXiv:2507.01079 [cs.DB] <https://arxiv.org/abs/2507.01079>
- [34] Percena. 2025. LoCoMo-MC10 · Long Conversation Memory Multiple-Choice 10. <https://huggingface.co/datasets/Percena/locomomo10>
- [35] Jeffrey Pound, Floris Chabert, Arjun Bhushan, Ankur Goswami, Anil Pacaci, and Shihabur Rahman Chowdhury. 2025. MicroNN: An On-device Disk-resident Updatable Vector Database. In *Companion of the 2025 International Conference on Management of Data (Berlin, Germany) (SIGMOD/PODS '25)*. Association for Computing Machinery, New York, NY, USA, 608–621.
- [36] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *ACM Trans. Storage* (Nov. 2025).
- [37] Llama Team. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI] <https://arxiv.org/abs/2407.21783>
- [38] Llama Team. 2024. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>.
- [39] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [40] Jun Wang, Yunxiang Yao, Wenwei Kuang, Runze Mao, Zhenhao Sun, Zhuang Tao, Ziyang Zhang, Dengyu Li, Jiajun Chen, Zhili Wang, Kai Cui, Congzhi Cai, Longwen Lan, and Ken Zhang. 2025. OmniInfer: System-Wide Acceleration Techniques for Optimizing LLM Serving Throughput and Latency. arXiv:2511.22481 [cs.DC] <https://arxiv.org/abs/2511.22481>
- [41] Tuowei Wang, Ruwen Fan, Minxing Huang, Zixu Hao, Kun Li, Ting Cao, Youyou Lu, Yaoxue Zhang, and Ju Ren. 2025. Neuralink: Fast on-Device LLM Inference with Neuron Co-Activation Linking. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 147–162.
- [42] Zhaode Wang, Jingbang Yang, Xinyu Qian, Shiwen Xing, Xiaotang Jiang, Chengfei Lv, and Shengyu Zhang. 2024. MNN-LLM: A Generic Inference Engine for Fast Large Language Model Deployment on Mobile Devices. In *Proceedings of the 6th ACM International Conference on Multimedia in Asia Workshops (MMAsia '24 Workshops)*. Association for Computing Machinery, New York, NY, USA, Article 11, 7 pages.
- [43] Jianyu Wei, Shijie Cao, Ting Cao, Lingxiao Ma, Lei Wang, Yanyong Zhang, and Mao Yang. 2025. T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge. In *Proceedings of the Twentieth European Conference on Computer Systems (Rotterdam, Netherlands) (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 278–292.
- [44] Jianyu Wei, Qingtao Li, Shijie Cao, Lingxiao Ma, Zixu Hao, Yanyong Zhang, Xiaoyan Hu, and Ting Cao. 2025. T-MAN: Enabling End-to-End Low-Bit LLM Inference on NPUs via Unified Table Lookup. arXiv:2511.11248 [cs.AR] <https://arxiv.org/abs/2511.11248>
- [45] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. SmoothQuant: accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML '23)*. JMLR.org, Article 1585, 13 pages.
- [46] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. 2026. Strata: Hierarchical Context Caching for Long Context Language Model Serving. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI' 26)*. USENIX Association, 1–16.
- [47] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. 2025. Fast On-device LLM Inference with NPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 445–462.
- [48] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. 2024. PowerInfer-2: Fast Large Language Model Inference on a Smartphone. arXiv:2406.06282 [cs.LG] <https://arxiv.org/abs/2406.06282>
- [49] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 2369–2380.
- [50] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems (Rotterdam, Netherlands) (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 94–109.
- [51] Lu Ye, Ze Tao, Yong Huang, and Yang Li. 2024. ChunkAttention: Efficient Self-Attention with Prefix-Aware KV Cache and Two-Phase Partition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL '24)*. Association for Computational Linguistics, Bangkok, Thailand, 11608–11620.

- [52] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. arXiv:2506.05176 [cs.CL] <https://arxiv.org/abs/2506.05176>
- [53] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '24). Curran Associates Inc., Red Hook, NY, USA, Article 2000, 27 pages.

A Graph Structure and Quantization Scheme of NPU Selective Recompute Graph

We follow the $W_{\text{INT4}}A_{\text{UINT16}}KV_{\text{INT8}}$ quantization adopted in ExecuTorch [12]. Specifically, KV tensors are quantized to 8-bit using SpinQuant [27], which applies a Hadamard transformation to mitigate the accuracy loss caused by severe outliers in KV distributions [27, 45]. On the other hand, the precision-critical KV deviation computation and Top-K selection are retained in FP16 precision, while KV tensors are SpinQuanted to INT8, and most intermediate activations use UINT16. The graph structure and corresponding bit-width is illustrated in Figure 18.

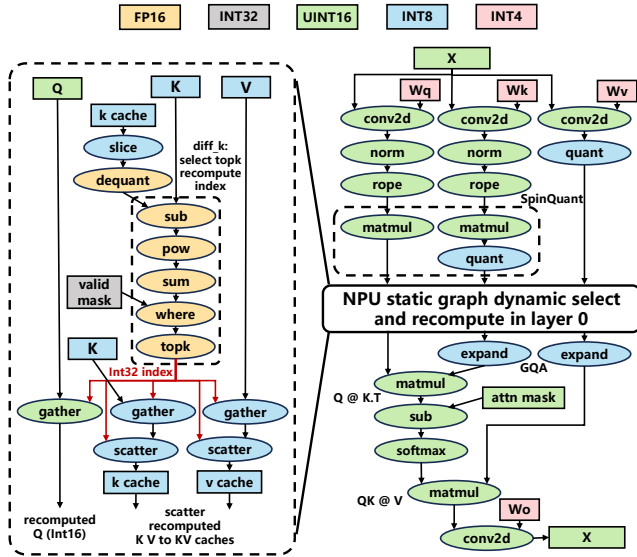


Figure 18. Quantized NPU Selective KV Recompute Graph of Layer 0.

B Chunk Merge Algorithm for Efficient Static Graph Utilization

We give the complete formulation of the chunk merge dynamic program.

Setup. Let the token sequence be partitioned into ordered segments

$$P_j = (s_j, e_j, p_j), \quad j = 0, \dots, m-1,$$

where $[s_j, e_j]$ is the span of segment j and $p_j \in \{0, 1, 2\}$ indicates prefix reuse, non-prefix reuse, and new tokens, respectively. Let L_s and L_p be the token capacity of the selective recomputation graph and prefill graph, and let t_s and t_p be their latency. Let $r \in (0, 1]$ be the minimum recomputation ratio.

We define $T[i]$ as the minimum latency to process tokens up to position i . The base case is that prefix reuse tokens require no computation, so if position i lies entirely in the prefix-reuse region, then $T[i] = 0$.

Transition. For a chunk ending at token position i , we consider two options:

$$T[i] = \min\{T[i - l_s(i)] + t_s, T[i - l_p(i)] + t_p\}. \quad (5)$$

Here:

- $l_s(i)$ is the longest valid suffix ending at i that can be packed into one selective recomputation call.
- $l_p(i)$ is the longest valid suffix ending at i that can be packed into one prefill call.

The algorithmic implementation revolves around this transition function, and the pseudo code is presented in Algorithm 1.

Computation of $l_p(i)$. The prefill graph simply packs as many trailing tokens as allowed by its capacity. Starting from token i and scanning backward over segments, we accumulate tokens until either: (1) the prefill budget L_p is exhausted, or (2) we reach the prefix reuse region. Thus, if the accumulated packed length is ℓ , then $l_p(i) = \ell$.

Computation of $l_s(i)$. The selective recomputation graph has capacity L_s , but new tokens may only occupy a bounded portion of that capacity. When scanning backward from token i :

- if the current segment is non-prefix reuse ($p_j = 1$), its tokens consume recomputation capacity directly;
- if the current segment is new ($p_j = 2$), only a bounded number of its tokens may be merged into the current recomputation call so that the recomputation ratio remains at least r .

Equivalently, if the remaining recomputation capacity is q , then at most $\lfloor qr \rfloor$ new tokens may be absorbed from the current new-token segment. If Δ new tokens are absorbed, they consume $\lceil \Delta/r \rceil$ units of recomputation capacity. Scanning stops when the recomputation capacity is exhausted or when the prefix reuse region is reached. If the total absorbed suffix length is ℓ , then $l_s(i) = \ell$.

The functions for computing $l_p(i)$ and $l_s(i)$ are presented in Algorithm 2 and 3 respectively.

Algorithm 1: Dynamic Programming Chunk Merge

Input: Segment list $P_j = (s_j, e_j, p_j)$ for $j = 0, \dots, m - 1$;
selective recomputation capacity L_s and latency t_s ;
prefill capacity L_p and latency t_p ;
minimum recomputation ratio r

Output: Minimum latency $T[L - 1]$ and chunk decisions

```

1  $L \leftarrow e_{m-1} + 1$ ;
2 Initialize  $T[i] \leftarrow +\infty$  and  $\text{dec}[i] \leftarrow \emptyset$  for all  $i = 0, \dots, L - 1$ ;
3 for  $i \leftarrow 0$  to  $L - 1$  do
4   if  $i$  is in the prefix reuse region then
5      $T[i] \leftarrow 0$ ;
6     continue;
7    $l_s \leftarrow \text{ComputeSRLength}(P, i, L_s, r)$ ;
8    $l_p \leftarrow \text{ComputePrefillLength}(P, i, L_p)$ ;
9    $v_s \leftarrow T[i - l_s] + t_s$ ; // use 0 if  $i - l_s < 0$ 
10   $v_p \leftarrow T[i - l_p] + t_p$ ; // use 0 if  $i - l_p < 0$ 
11  if  $v_s < v_p$  then
12     $T[i] \leftarrow v_s$ ;
13     $\text{dec}[i] \leftarrow (i - l_s, \text{SR})$ ;
14  else
15     $T[i] \leftarrow v_p$ ;
16     $\text{dec}[i] \leftarrow (i - l_p, \text{P})$ ;
17 return  $T[L - 1]$  and  $\text{dec}$ ;
```

Algorithm 2: ComputePrefillLength(P, i, L_p)

Input: Segment list P , ending position i , prefill budget L_p

Output: $l_p(i)$

```

1 Locate the segment index  $j$  such that  $i \in [s_j, e_j]$ ;
2  $q \leftarrow L_p, \ell \leftarrow 0, x \leftarrow i$ ;
3 while  $q > 0$  and segment  $j$  is not prefix reuse do
4    $a \leftarrow x - s_j + 1$ ;
5    $\Delta \leftarrow \min(q, a)$ ;
6    $\ell \leftarrow \ell + \Delta$ ;
7    $q \leftarrow q - \Delta$ ;
8    $x \leftarrow x - \Delta$ ;
9   if  $x < s_j$  then
10     $j \leftarrow j - 1$ ;
11 return  $\ell$ ;
```

C Fast LCS for Non-prefix Matching

To match the reusable substrings in candidate cached chunks, we apply a linear-time LCS (Longest Common Substring) for fast matching.

To compute the longest common substring between two token sequences, we use a suffix automaton with sparse

Algorithm 3: ComputeSRLength(P, i, L_s, r)

Input: Segment list P , ending position i , recomputation budget L_s , ratio r

Output: $l_s(i)$

```

1 Locate the segment index  $j$  such that  $i \in [s_j, e_j]$ ;
2  $q \leftarrow L_s, \ell \leftarrow 0, x \leftarrow i$ ;
3 while  $q > 0$  and segment  $j$  is not prefix reuse do
4    $a \leftarrow x - s_j + 1$ ; // available suffix length in current segment
5   if  $p_j = 2$  then
6      $\Delta \leftarrow \min(\lfloor qr \rfloor, a)$ ;
7     if  $\Delta = 0$  then
8       break;
9      $\ell \leftarrow \ell + \Delta$ ;
10     $q \leftarrow q - \lceil \Delta / r \rceil$ ;
11  else
12     $\Delta \leftarrow \min(q, a)$ ;
13     $\ell \leftarrow \ell + \Delta$ ;
14     $q \leftarrow q - \Delta$ ;
15     $x \leftarrow x - \Delta$ ;
16    if  $x < s_j$  then
17       $j \leftarrow j - 1$ ;
18 return  $\ell$ ;
```

transitions. Given two sequences X and Y , where each sequence contains at most 512 tokens (database chunk size limit) and the token vocabulary can be as large as 10^6 , we build the suffix automaton over the shorter sequence X and then stream the longer sequence Y through the automaton. During the scan, we maintain the current automaton state and the length of the current match; when the next token transition is absent, we follow suffix links until a valid transition is found or the root is reached. The maximum matched length observed during this scan is the longest common token substring length.

The key design choice is to avoid alphabet-dense transition tables. Although the global vocabulary is large, each sequence contains only a small number of tokens, and a suffix automaton over X contains at most $2|X| - 1$ states and $O(|X|)$ transitions. Therefore, we store outgoing transitions sparsely as token-id-to-state mappings, making the memory usage independent of the global vocabulary size. With hashtable transitions, transition lookup takes expected $O(1)$ time, so the algorithm runs in expected $O(|X| + |Y|)$ time and uses $O(|X|)$ space. For a deterministic worst-case implementation, hash tables can be replaced by sorted sparse transition arrays or balanced maps, giving $O((|X| + |Y|) \log d_{\max})$ time and $O(|X|)$ space, where $d_{\max} \leq |X|$ is the maximum outgoing degree of any automaton state. Since $|X|, |Y| \leq 512$, the automaton has at most 1023 states and $\log d_{\max} \leq 9$, so the

deterministic implementation remains approximately linear in practice. The pseudo code is presented in Algorithm 4.

Algorithm 4: Longest Common Token Substring

Input: Token sequences A and B

Output: Longest common token substring length

```

1 if  $|A| > |B|$  then
2   swap  $A$  and  $B$ ;
3 Build a suffix automaton  $S$  over  $A$  with sparse token
  transitions;
4  $v \leftarrow S.root$ ;  $\ell \leftarrow 0$ ; best  $\leftarrow 0$ ;
5 foreach token  $x \in B$  do
6   while  $v \neq S.root$  and  $x \notin S.next[v]$  do
7      $v \leftarrow S.link[v]$ ;
8      $\ell \leftarrow S.len[v]$ ;
9   if  $x \in S.next[v]$  then
10     $v \leftarrow S.next[v][x]$ ;
11     $\ell \leftarrow \ell + 1$ ;
12   else
13     $v \leftarrow S.root$ ;
14     $\ell \leftarrow 0$ ;
15   best  $\leftarrow \max(\text{best}, \ell)$ ;
16 return best;

```

D Flash-Storage SQLite-Based DB Organization

Directly storing large KV tensors as SQLite BLOB objects can lead to severe internal and external fragmentation, especially under frequent insertion, eviction, and update operations. Fragmentation further degrades insertion efficiency because free pages become sparsely scattered, making it difficult for SQLite to allocate sufficiently large contiguous regions for KV BLOBs. Moreover, since SQLite page management itself is already built on top of the underlying file system, using SQLite to manage large KV tensors introduces another unnecessary storage-management layer and additional overhead. In practice, both insertion and loading latency can increase to the order of seconds if KV storage is handled entirely by SQLite.

Therefore, as illustrated in Figure 19, instead of embedding KV tensors directly inside SQLite rows as large BLOB objects, we organize flash storage using a lightweight SQLite metadata index combined with external file-system storage. Specifically, SQLite stores only compact metadata entries, including chunk keys, prompt hashes, locality statistics, and file paths, while the actual KV tensors are serialized as separate files in the underlying file system. This design avoids repeated large-BLOB reallocations within SQLite pages, substantially reducing fragmentation and write amplification. It also improves eviction and compaction efficiency, since removing a KV chunk only requires deleting its metadata

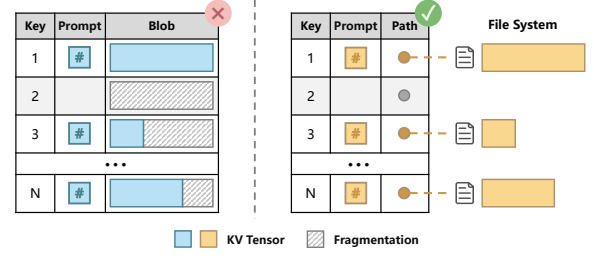


Figure 19. KV tensors blob storage design (self-managed outside SQLite).

entry and corresponding file without triggering large-scale database-page reorganization. As a result, the storage system maintains stable lookup efficiency while supporting scalable GB-level KV persistence on resource-constrained mobile flash storage.

E Locality Model for Prefetch and Reuse

The locality model estimates the probability that a KV chunk c_i will be reused in the near future. The resulting locality score is used to guide both prefetch and eviction decisions across the memory hierarchy.

Conventional cache policies such as LRU and LFU only capture temporal locality, while prefix-aware methods such as PGDSF in RAGCache [21] mainly target prefix reuse and cannot effectively model non-prefix reuse behaviors and their associated recomputation costs. To address this limitation, we propose a lightweight locality model tailored for mobile long-context workloads, together with an online eviction-cost estimator that jointly considers both prefix and non-prefix reuse.

For a cached chunk c_i , the overall locality score is defined as:

$$R(c_i) = w_s R_s(c_i) + w_t R_t(c_i) + w_m R_m(c_i), \quad (6)$$

where R_s , R_t , and R_m denote spatial, temporal, and semantic locality scores, respectively, and R_s , R_t , R_m are normalization weights.

The three locality dimensions are defined as follows:

- **Spatial locality** $R_s(c_i)$ captures whether a chunk belongs to the same prefix/non-prefix tree or document as recently reused chunks. This is motivated by the observation that recently accessed documents and histories are more likely to be revisited in the near future.
- **Temporal locality** $R_t(c_i)$ models both recency and access frequency through a lightweight combination of LRU and LFU signals, allowing frequently accessed chunks to remain in higher memory levels while gradually evicting colder entries.
- **Semantic locality** $R_m(c_i)$ models correlations among reusable chunks using a lightweight co-access lookup table. Chunks that frequently co-occur in prompts are assigned higher semantic locality scores. For example,

SMS-related skills are often co-accessed with website-login skills for verification-code handling, and therefore exhibit strong semantic locality.

The eviction policy considers all three locality dimensions, while the prefetch policy only uses spatial and semantic locality to avoid unnecessary memory pollution.

F Recomputation/Reloading Cost Model

The eviction cost model estimates the future overhead introduced by removing a KV chunk from different storage levels. Since chunks stored in the CPU memory pool and flash storage incur fundamentally different recovery costs, we model them separately.

CPU memory-pool eviction cost. For a chunk c_i stored in the CPU memory pool, the eviction cost mainly corresponds to the future reload latency from flash storage:

$$C_{\text{cpu}}(c_i) = \begin{cases} T_{\text{load}}(c_i), & c_i \in \mathcal{P} \\ 0, & c_i \in \mathcal{N} \end{cases}, \quad (7)$$

where $\mathcal{P}$ and $\mathcal{N}$ denote the sets of prefix-reusable and non-prefix-reusable chunks, respectively, and $T_{\text{load}}(c_i)$ is the flash to memory loading latency of chunk c_i . For non-prefix-reusable chunks, the loading latency can largely be hidden through compute-storage overlap, making the effective eviction cost close to zero.

Flash-storage eviction cost. The eviction cost of a chunk stored in flash involves 2 terms: the first is the recomputation cost of the evicted chunk itself, while the second is the additional overhead derived from the cascading effect of breaking the prefix reuse chain.

Specifically, the primary component is the recomputation cost $\text{Latency}_P(|c_i|)$, denoting the full prefill latency of the chunk estimated in Section 4.1.

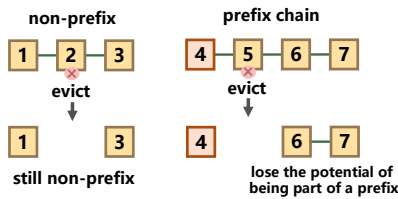


Figure 20. Evicting a chunk on prefix chain makes subsequent chunks lose the potential of being part of a prefix.

Besides recomputation cost of the evicted chunk itself, evicting a chunk on a prefix path may cause its descendant chunks to lose prefix reusability and fall back to non-prefix reuse and bring additional cascading cost C_{cascade} . As illustrated in Figure 20, chunks 6 and 7 can originally be reused as part of the prefix chain 4567. However, evicting chunk 5 breaks the prefix chain and turns chunks 6 and 7 into pure non-prefix chunks, thereby increasing future recomputation overhead. In contrast, evicting chunk 2 introduces no

additional cost to chunk 3 because it is already non-prefix-reusable. The cascading cost is formulated as:

$$C_{\text{cascade}} = \sum_{c_j \in \text{Des}(c_i)} f(c_j) \cdot \text{Latency}_S(|c_i|), \quad (8)$$

where $\text{Des}(c_i)$ denotes the descendant chunks of c_i on the prefix tree, $f(c_j)$ is the prefix-hit frequency factor estimating the likelihood of future prefix reuse, $\text{Latency}_S(|c_i|)$ is the additional selective recomputation graph execution overhead caused by turning from prefix to non-prefix.

Combining the 2 terms, the full eviction cost formulate as:

$$C_{\text{flash}}(c_i) = \text{Latency}_P(|c_i|) + C_{\text{cascade}}(c_i). \quad (9)$$

G Cost-Aware Prefetch and Eviction

G.1 Prefetch

The prefetch policy proactively loads reusable chunks whose locality scores $R(c)$ greater or equal to a threshold from flash storage into memory to reduce future loading latency. Prefetching is performed asynchronously during idle I/O periods and primarily targets prefix-reusable chunks with high predicted reuse probability.

G.2 Eviction

The eviction policy ranks chunks according to the locality-cost product: $R(c) \cdot C(c)$. Chunks with high locality and high recovery cost are preferentially retained in higher memory levels, while less useful chunks are demoted or removed. This policy jointly optimizes cache hit rate and recovery overhead under constrained on-device memory and storage capacity.